Intrusion Detection with Decision-Tree Learning

CSC 7210

1. Overview

I implemented the DECISION-TREE-LEARNING algorithm from Russell and Norvig in a Python notebook and used it to classify network connections from the UNSW data set provided for the project (211,043 records, 43 attributes, and a class column with normal traffic and nine attack types: backdoor, DDoS, DoS, injection, MITM, password, ransomware, scanning and XSS). The zip contains a single CSV, so the notebook first creates the three inputs the program requires: a stratified 70/30 split (147,730 training and 63,313 test records, fixed random seed) and an attribute description file computed from the training set only. The program, main(attribute_file, training_file, test_file), prints the tree in pre-order with deeper nodes indented further, followed by the training and test accuracy. No machine-learning library was used; NumPy is used only for counting and pandas only for reading CSV files.

2. Experiences Creating the Decision-Tree Learner

Turning real data into discrete attributes. The pseudocode makes one branch per attribute value, so every attribute needs a small, known set of values, and most of the design work went into the attribute description file rather than the recursion itself. Continuous attributes (duration, byte and packet counts) are split into up to ten equal-frequency bins whose cut points are stored in the file. Ports look numeric but are really identifiers, so they are treated as discrete, keeping the 25 most frequent values plus an “other” value; long text fields such as dns_query and http_uri are handled the same way. I ignored src_ip and dst_ip because they identify machines in the test bed and would let the tree memorise hosts, and I ignored label because it gives the answer away (label = 0 exactly when type = normal).

Efficiency: A direct translation that copies lists of examples at every node would be slow on 150,000 records. Instead, every value is encoded once as a small integer, each node only keeps the row indices of its examples, and information gain is computed from a value-by-class count table built with one bincount call. With this, learning the full tree takes about 3 seconds on my laptop and 8 seconds on Cloudbank.

Surprises in the output: Following the pseudocode exactly, a node gets a branch for every value of its attribute, even values with no training examples. 55,591 of the 57,045 leaves are such empty branches, so I print them as a single “any of the other k values” line per node; otherwise the printed tree would have been 66,000 lines long. The tree also always reaches depth 40, the number of attributes. The reason is that some records have identical attribute values but different classes: no attribute can separate them, yet the algorithm keeps splitting with zero information gain until the attributes run out. Finally, on Cloudbank the printed tree exceeded Jupyter’s output rate limit (“IOPub data rate exceeded”) and was cut from the notebook, so the program now prints the tree in chunks and also writes it to program_output.txt.

3. Performance of the Decision-Tree Learner

Experiment (70/30 stratified split)	Training acc.	Test acc.
Multi-class tree (class = type, 10 classes)	98.15%	97.81%
Test records never seen in training (55,952)	N/A	97.55%
Test records duplicated in training (7,361)	N/A	99.81%
Same tree, attack vs. normal only	N/A	99.42%
Binary tree (class = label, 0/1)	99.53%	99.43%
Multi-class, stop at zero information gain	98.15%	97.81%

Table 1. Accuracy of the learned trees (all numbers from the Cloudbank run).

Accuracy: The multi-class tree reaches 98.15% on the training set and 97.81% on the unseen test set. The small gap (0.3 points) shows little overfitting, which is notable because the tree is unpruned. Training accuracy never reaches 100% because about 1.9% of training records are indistinguishable from records of another class once their numeric values are binned. Because the data contains 20,569 duplicate records, 11.6% of the test records also appear in the training set; on the 55,952 truly unseen records the accuracy is 97.55%, so the duplicates inflate the headline figure only slightly.

As an intrusion detector: If every predicted attack type counts as an alarm, the same tree catches 99.36% of attacks (309 missed) with a false-alarm rate of 0.39% (59 of 15,000 normal connections). Training a tree directly on the binary label gives a similar 99.43%. Backdoor, ransomware and scanning are recognised almost perfectly (recall above 99%). The main errors are between related attacks: 250 DoS connections were labelled normal, and injection was confused with XSS (180) and password attacks (166), which is plausible because all three are web-application traffic on similar ports. MITM, with only 313 test records, is the weakest class (recall 0.85, precision 0.58), since the majority-value rule favours the large classes.

Learning curve and tree size: With only 1% of the training data (1,477 records) the tree already reaches 94.4% test accuracy, 97.1% with 10%, and 97.8% with all of it (Figure 1). Learning time grows roughly linearly, from 0.3 s to 8.0 s for 100 times more data, while the tree grows from 302 to 9,043 internal nodes. Adding a single rule that stops when the best information gain is zero produces a tree with 692 internal nodes and depth 12 instead of 9,043 and 40, with identical training and test accuracy and a quarter of the learning time. In other words, more than 90% of the textbook tree is splitting that contributes nothing.

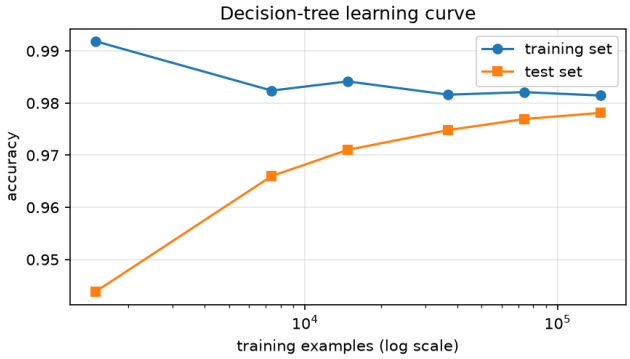


Figure 1. Training and test accuracy as the training set grows (log scale).

Knowledge representation: The tree is readable as a rule base, which is the main appeal of this learner for an IDS. The root tests dst_port (port 80 alone routes 49,578 records, mostly injection, XSS and password attacks), followed by src_ip_bytes, conn_state and dns_query, which matches how an analyst would separate web attacks, scans and DNS-based traffic. The weaknesses are equally visible: information gain favours attributes with many values, which partly explains why the 26-valued dst_port is chosen at the root (gain ratio would correct this); fixed bins lose the exact thresholds that a C4.5-style binary split would find; and without pruning the full tree is far too large to read as a whole.

4. Use of NAIRR Resources (Cloudbank)

4.1 Performance and Computational Advantages

The final run used the Cloudbank Python 3.13 kernel on a Linux container (NumPy 2.4.6, pandas 3.0.3). Loading took 4.0 s, learning 7.9 s and evaluation 0.6 s, against 1.9 s, 2.8 s and 0.3 s on my laptop, and the whole notebook ran in 70 seconds. The notebook read the container limits directly: 2 CPUs and 4 GB of memory (more than the 1 CPU and 2 GB listed in the hub documentation). Because the learner is single-threaded, the extra CPU does not help, and for this workload Cloudbank was not faster than a modern laptop. Its advantage was a ready-made, identical environment: NumPy, pandas and matplotlib were already installed, nothing had to be configured, and the run did not depend on my own machine. Peak memory of the notebook was 729 MB, less than a fifth of the allocation.

4.2 Potential for Large-Scale Analysis

The learner scales roughly linearly with the number of records, so the 211,000-record data set is well within reach. The full UNSW collection (about 100 GB of raw packet captures) would not be with 4 GB of memory and 10 GB of storage, the data would have to be processed in parts, or run on the larger CPU and GPU allocations that NAIRR makes available through its other resource providers. Because the work lives in a standard notebook, moving it to a bigger allocation would require no code changes.

4.3 Support for Advanced Modeling

The assignment requires implementing the tree from scratch, so no modelling libraries were used. The Cloudbank image does, however, include scikit-learn, PyTorch, SciPy and statsmodels, so the same notebook could be extended to compare this tree with random forests, gradient boosting or neural networks, or to add pruning and gain-ratio experiments, without installing anything.

4.4 Ease of Use and Workflow Integration

Logging in with my @tntech.edu account opens JupyterLab in the browser, and after uploading the notebook and the data file the whole analysis runs with Restart Kernel and Run All in about a minute. I developed the notebook on my own machine and then ran it unchanged on Cloudbank, which worked because both use standard Jupyter. Results files persist in the home directory and the notebook can be exported directly for submission. The one problem was the output rate limit, which dropped the printed tree from my first run; after changing the program to print in chunks, the second run displayed the full tree. It was easy to fix once understood, but a new user might not recognise the message.

4.5 Overall Perceptions

Cloudbank was a convenient, zero-setup place to run and document the project, and it gives every student the same environment, which makes results reproducible. For a project of this size its value is accessibility and consistency rather than speed; the small per-user allocation means it is a good starting point, and larger NAIRR resources would be needed for full-scale intrusion-detection data.

5. Conclusion

The textbook decision-tree learner, implemented in about 200 lines of Python, detects attacks in this data set with 97.8% multi-class accuracy and 99.4% attack-versus-normal accuracy, and learns from 150,000 records in seconds. The main lessons were that preparing discrete attributes matters as much as the algorithm, that the pure algorithm builds far larger trees than necessary, and that a simple zero-gain stopping rule removes over 90% of the tree at no cost in accuracy.